U24AI006

ASSIGNMENT -08

Use the following graph for the Traveling Salesperson problem. Find the least cost path in the graph for the TSP problem.

	A	B	C	D	E	F	G	H	
A	[0,	10,	15,	20,	25,	30,	35,	40],	# City 0
B	[12,	0,	35,	15,	20,	25,	30,	45],	# City 1
C	[25,	30,	0,	10,	40,	20,	15,	35],	# City 2
D	[18,	25,	12,	0,	15,	30,	20,	10],	# City 3
E	[22,	18,	28,	20,	0,	15,	25,	30],	# City 4
F	[35,	22,	18,	28,	12,	0,	40,	20],	# City 5
G	[30,	35,	22,	18,	28,	32,	0,	15],	# City 6
H	[40,	28,	35,	22,	18,	25,	12,	0]	# City 7

1. Use the local beam search to solve the problem. Give a comparative analysis of the algorithm when the beam width k=3,5,10. Does the convergence depend on the value of k?
2. Use the genetic algorithm to solve the problem. Use one crossover point and two crossover points to generate the offspring. Does the number of crossover points impact the convergence rate?

CODE(1):

```
import random

# ── Distance matrix (8 cities: A-H) ──────────────────────────
DIST = [
    [0,  10, 15, 20, 25, 30, 35, 40],   # A
    [12,  0, 35, 15, 20, 25, 30, 45],   # B
    [25, 30,  0, 10, 40, 20, 15, 35],   # C
    [18, 25, 12,  0, 15, 30, 20, 10],   # D
    [22, 18, 28, 20,  0, 15, 25, 30],   # E
    [35, 22, 18, 28, 12,  0, 40, 20],   # F
    [30, 35, 22, 18, 28, 32,  0, 15],   # G
    [40, 28, 35, 22, 18, 25, 12,  0],   # H
]
CITIES = list(range(len(DIST)))
CITY_NAMES = "ABCDEFGH"


def tour_cost(tour):
    """Total round-trip cost for a given tour (list of city indices)."""
    cost = sum(DIST[tour[i]][tour[i + 1]] for i in range(len(tour) - 1))
    cost += DIST[tour[-1]][tour[0]]        # return to start
    return cost


def random_tour():
    """Generate a random permutation of all cities."""
```

```
    t = list(CITIES)
    random.shuffle(t)
    return t


def get_neighbours(tour):
    """Generate neighbours by swapping every pair of cities."""
    neighbours = []
    n = len(tour)
    for i in range(n):
        for j in range(i + 1, n):
            nb = list(tour)
            nb[i], nb[j] = nb[j], nb[i]
            neighbours.append(nb)
    return neighbours


# ── Local Beam Search ──────────────────────────────────────────
def local_beam_search(k, max_iter=500, seed=42):
    """
    Local beam search for TSP.
    k        : beam width (number of states kept at each step)
    Returns : (best_tour, best_cost, history_of_best_cost_per_iter, iterations)
    """
    random.seed(seed)

    # 1. Start with k random tours
    beams = [random_tour() for _ in range(k)]
    best_tour = min(beams, key=tour_cost)
    best_cost = tour_cost(best_tour)
    history = [best_cost]

    for it in range(1, max_iter + 1):
        # 2. Generate ALL neighbours of all k states
        all_neighbours = []
        for tour in beams:
            all_neighbours.extend(get_neighbours(tour))

        # 3. Pick the k best neighbours
        all_neighbours.sort(key=tour_cost)
        beams = all_neighbours[:k]

        # Track the best so far
        current_best = beams[0]
        current_cost = tour_cost(current_best)
```

```
        if current_cost < best_cost:
            best_cost = current_cost
            best_tour = list(current_best)

        history.append(best_cost)

        # 4. Convergence check — stop if no improvement for 20 steps
        if len(history) > 20 and history[-1] == history[-21]:
            break

    return best_tour, best_cost, history, it


def format_tour(tour):
    return " -> ".join(CITY_NAMES[c] for c in tour) + " -> " + CITY_NAMES[tour[0]]


# ── Run & Compare k = 3, 5, 10 ──────────────────────────────
if __name__ == "__main__":
    print("TSP — Local Beam Search (8 cities A-H)\n")
    print(f"{'k':>4} | {'Cost':>6} | {'Iters':>5} | Best Tour")
    print("-" * 55)

    results = {}
    for k in [3, 5, 10]:
        tour, cost, history, iters = local_beam_search(k)
        results[k] = (tour, cost, history, iters)
        print(f"{k:>4} | {cost:>6} | {iters:>5} | {format_tour(tour)}")

    print(f"\nDoes convergence depend on k?")
    costs = {k: results[k][1] for k in results}
    if costs[3] == costs[5] == costs[10]:
        print("All beam widths converged to the same cost — for this small")
        print("problem even k=3 suffices. Larger k helps on bigger instances.")
    else:
        print("Yes — wider beams find better solutions by exploring more")
        print("neighbours per iteration, avoiding local minima.")
```

U24AI006

OUTPUT:

```
QUESTION 1.py
TSP — Local Beam Search (8 cities A–H)

   k |   Cost | Iters | Best Tour
-------------------------------------------------------
   3 |    121 |    25 | C -> D -> H -> G -> F -> E -> B -> A -> C
   5 |    121 |    24 | D -> H -> G -> F -> E -> B -> A -> C -> D
  10 |    121 |    24 | D -> H -> G -> F -> E -> B -> A -> C -> D

Does convergence depend on k?
All beam widths converged to the same cost — for this small
problem even k=3 suffices. Larger k helps on bigger instances.
```

CODE(2):

```
import random

# ── Distance matrix (8 cities: A-H) ──────────────────────────
DIST = [
    [0,  10, 15, 20, 25, 30, 35, 40],   # A
    [12,  0, 35, 15, 20, 25, 30, 45],   # B
    [25, 30,  0, 10, 40, 20, 15, 35],   # C
    [18, 25, 12,  0, 15, 30, 20, 10],   # D
    [22, 18, 28, 20,  0, 15, 25, 30],   # E
    [35, 22, 18, 28, 12,  0, 40, 20],   # F
    [30, 35, 22, 18, 28, 32,  0, 15],   # G
    [40, 28, 35, 22, 18, 25, 12,  0],   # H
]
NUM_CITIES = len(DIST)
CITY_NAMES = "ABCDEFGH"


def tour_cost(tour):
    """Total round-trip cost."""
    cost = sum(DIST[tour[i]][tour[i + 1]] for i in range(len(tour) - 1))
    cost += DIST[tour[-1]][tour[0]]
    return cost


def format_tour(tour):
    return " -> ".join(CITY_NAMES[c] for c in tour) + " -> " + CITY_NAMES[tour[0]]


# ── GA helpers ────────────────────────────────────────────────
def random_tour():
    t = list(range(NUM_CITIES))
    random.shuffle(t)
```

```
    return t


def fitness(tour):
    """Lower cost = higher fitness.  We use inverse cost."""
    return 1.0 / tour_cost(tour)


def select_parent(population, fitnesses):
    """Roulette-wheel (fitness-proportionate) selection."""
    total = sum(fitnesses)
    pick = random.uniform(0, total)
    cumulative = 0
    for ind, f in zip(population, fitnesses):
        cumulative += f
        if cumulative >= pick:
            return ind
    return population[-1]


# ── Crossover operators (order-based for permutations) ──────────
def order_crossover_1pt(p1, p2):
    """Single-point Order Crossover (OX1 variant).
    Pick one crossover point; copy left segment from p1,
    fill remaining cities in the order they appear in p2."""
    n = len(p1)
    cx = random.randint(1, n - 1)          # crossover point
    child = p1[:cx]
    for gene in p2:
        if gene not in child:
            child.append(gene)
    return child


def order_crossover_2pt(p1, p2):
    """Two-point Order Crossover (OX1).
    Copy the segment between two points from p1,
    fill the rest from p2 preserving order."""
    n = len(p1)
    cx1, cx2 = sorted(random.sample(range(1, n), 2))   # two crossover points
    child = [None] * n
    child[cx1:cx2] = p1[cx1:cx2]
    fill = [g for g in p2 if g not in child[cx1:cx2]]
    idx = 0
    for i in range(n):
```

```
        if child[i] is None:
            child[i] = fill[idx]
            idx += 1
    return child


def mutate(tour, rate=0.1):
    """Swap mutation: with probability `rate`, swap two random cities."""
    if random.random() < rate:
        i, j = random.sample(range(len(tour)), 2)
        tour[i], tour[j] = tour[j], tour[i]
    return tour


# ── Genetic Algorithm ──────────────────────────────────────────
def genetic_algorithm(crossover_fn, label, pop_size=50,
                      generations=300, mutation_rate=0.15, seed=42):
    """
    Run GA for TSP.
    crossover_fn : function(parent1, parent2) -> child
    Returns      : (best_tour, best_cost, history)
    """
    random.seed(seed)

    # Initialise population
    population = [random_tour() for _ in range(pop_size)]
    best_tour = min(population, key=tour_cost)
    best_cost = tour_cost(best_tour)
    history = [best_cost]

    for gen in range(1, generations + 1):
        fitnesses = [fitness(ind) for ind in population]

        new_pop = []
        # Elitism: carry forward the best individual
        new_pop.append(list(best_tour))

        while len(new_pop) < pop_size:
            p1 = select_parent(population, fitnesses)
            p2 = select_parent(population, fitnesses)
            child = crossover_fn(p1, p2)
            child = mutate(child, mutation_rate)
            new_pop.append(child)

        population = new_pop
```

```
        # Update best
        gen_best = min(population, key=tour_cost)
        gen_cost = tour_cost(gen_best)
        if gen_cost < best_cost:
            best_cost = gen_cost
            best_tour = list(gen_best)

        history.append(best_cost)

    return best_tour, best_cost, history


# ── Run & Compare ──────────────────────────────────────────────
if __name__ == "__main__":
    print("TSP — Genetic Algorithm (8 cities A-H)\n")

    tour1, cost1, hist1 = genetic_algorithm(order_crossover_1pt, "1-pt")
    tour2, cost2, hist2 = genetic_algorithm(order_crossover_2pt, "2-pt")

    first_hit_1 = next(i for i, c in enumerate(hist1) if c == cost1)
    first_hit_2 = next(i for i, c in enumerate(hist2) if c == cost2)

    print(f"{'Crossover':>10} | {'Cost':>6} | {'Converged @':>11} | Best Tour")
    print("-" * 65)
    print(f"{'1-Point':>10} | {cost1:>6} | {'Gen '+str(first_hit_1):>11} |
{format_tour(tour1)}")
    print(f"{'2-Point':>10} | {cost2:>6} | {'Gen '+str(first_hit_2):>11} |
{format_tour(tour2)}")

    print(f"\nDoes the number of crossover points impact convergence?")
    if first_hit_2 < first_hit_1:
        print(f"Yes — 2-point crossover converged faster (gen {first_hit_2} vs
{first_hit_1})")
        print("as it preserves larger sub-tour segments, adding genetic diversity.")
    elif first_hit_1 < first_hit_2:
        print(f"1-point converged faster here (gen {first_hit_1} vs
{first_hit_2}).")
    else:
        print("Both converged at the same generation for this instance.")
```

U24AI006

OUTPUT:

```
TSP — Genetic Algorithm (8 cities A–H)

 Crossover |   Cost | Converged @ | Best Tour
-----------------------------------------------------------------
   1-Point |    121 |      Gen 59 | C -> D -> H -> G -> F -> E -> B -> A -> C
   2-Point |    121 |      Gen 14 | D -> H -> G -> F -> E -> B -> A -> C -> D

Does the number of crossover points impact convergence?
Yes — 2-point crossover converged faster (gen 14 vs 59)
as it preserves larger sub-tour segments, adding genetic diversity.
```